20260115

sookmyung university ai vision lab

# LottieGPT

## :Tokenizing Vector Animation for Autoregressive Generation


STUDENT

Won Jiwoo

AUTHOR

Junhao Chen

# abstract

“first framework for **tokenizing** and **autoregressively generating vector animations**”

1. adopt **Lottie**, a widely deployed JSON-based animation standard

2. design a **Lottie Tokenizer**
   → reduce sequence length while preserving structural fiderlity

3. construct **LottieAnimation-660K,** the largest and most diverse vector animation dataset to date

4. **fintune Qwen-VL** to create **LottieGPT**

→ exibits strong generalization across diverse animation styled

→ outperforms previos state-of-the art models on SVG generation

# Introduction

existing T2V model can’t generate vector animation

VLM are incresingly capable of manipulating symbolic structures instead of only pixel grids

→ native vector animation generation could be within reach!

## two fundamental challenges

1. **tokenizer problem**

- vector animations : **hierarchical** structure + **time-dependent** transformation logic

=> ‘design the first **Lottie Tokenize**r’

(capture geometric primitives, hierarchical grouping, animated property curves, and interpolation settings.)

2. **data problem**

- no large-scale vector animation dataset existed

=> build the first Lottie animation corpus, **Lottie-660K**

(consisting of 660K high-quality, diverse animations exported from *After Effects* via the Bodymovin pipeline)

# Introduction

## contribution

1. present the first **framework for native vector animation generation**

2. design the first **tokenizer**

- ‘hierarchical geometric primitives, transforms, keyframes, and temporal dynamics’ → ‘a compact token sequence’.

3. construct **660K Lottie animation** and **15M Lottie static graphics** and propose **LottieBench**, the first benchmark for vector animation.

4. develop and train the first multimodal model (**LottieGPT**) for autoregressive vector animation generation,

- demonstrating strong in-the-wild performance and achieving new SOTA on SVG generation.

# Related work

## Pixels based Image and Video Generation

- Diffusion models : impressive visual quality

→ pixel-based approaches (non-editable fixed-resolution outputs, substantial storage, quality degradation, lack semantic editability)

- recent editing methods

→ iterative refinement and precise control

=> Our work shifting the generation paradigm from pixel space to **structured vector animation space.**

## Structured Data Generation

- In 3D generation

→ generate mesh structures as tokenized sequences through autoregressive modeling.
ex. LLaMA-Mesh, MeshGPT, and EdgeRunner

- In 2D vector graphics

→ generate SVG code by treating it as a code synthesis task
ex. StarVector, OmniSVG

→ remain confined to **static outputs**, **lacking temporal modeling capabilities**

# Related work

## Vector Graphics and Animation Generation

- optimization-based methods

(-) suffer from lengthy computation

- VLM based methods

(+) achieve better editability by generating SVG code directly

(-) confined to static outputs

### three challenges

1. limited format expressiveness
2. data scarcity
3. temporal coordination complexity

=> We address these limitations using the **Lottie format**

# SVG <-> Lottie

| 항목 | SVG | Lottie |
|---|---|---|
| 기본 목적 | 정적 벡터 그래픽 표현 | 벡터 기반 애니메이션 표현 |
| 파일 형식 | XML | JSON |
| 핵심 개념 | “무엇을 그릴까” | “어떻게 움직일까” |
| Animation 지원 | 기본적으로 없음 | Animation 자체가 핵심 |
| 애니메이션 구현 방식 | CSS / JavaScript / <animate> 태그 등을 별도로 추가해야 함 | Keyframe, easing, transform이 포맷 내부에 포함됨 |
| 구조 | Shape 중심 (path, circle) | Layer + Timeline + Keyframe 중심 |
| 시간(Timeline) 개념 | 제한적 | 내장 |

# Lottie

- Adobe After Effects(AE) : 영상/모션그래픽 만드는 전문 프로그램

→ 도형(layer)에 시간에 따른 변화(keyframe)를 지정

- Bodymovin

→ AE 프로젝트를 읽어서 Animation 정보를 JSON으로 변환 ("움직임 규칙"을 저장)

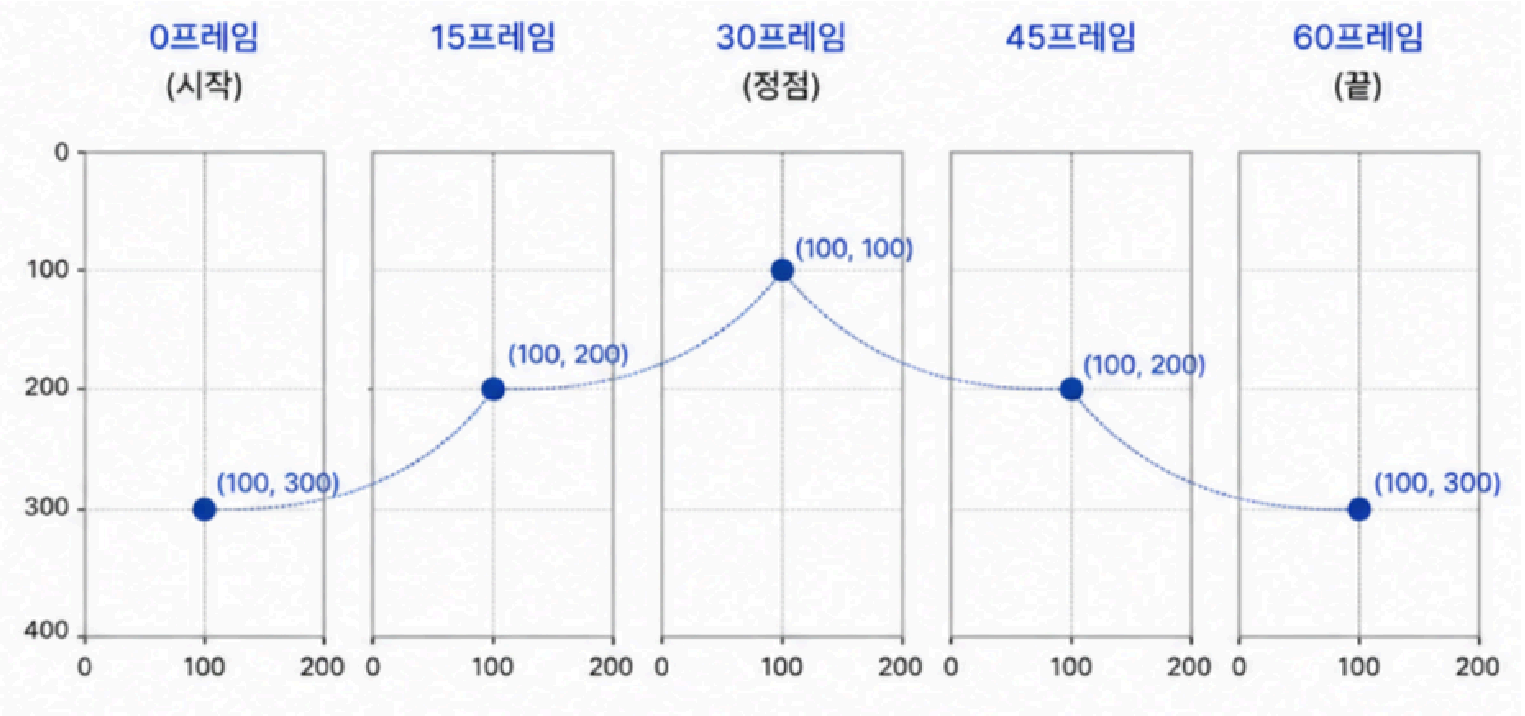
**AE**

| 시간 | 위치 |
|---|---|
| 0프레임 | 아래 |
| 30프레임 | 위 |
| 60프레임 | 아래 |

**Bodymovin**

**Lottie**

```
{
 "position": [
 {"t":0, "v":[100,300]},
 {"t":30, "v":[100,100]},
 {"t":60, "v":[100,300]}
 ]
}
```



# Data Curation Pipeline

SVG → limited support for complex timeline-based animations

=> **After Effects(AE) ecosystem**

- leveraging the Bodymovin plugin (AE animations → Lottie JSON format)

*Lottie : lightweight, cross-platform representation supporting keyframes, path morphing, and color gradients

## dataset

- AE source files → Lottie representation (via an automated pipeline = BodyMovin)
- 15M static vector graphics → Lottie representation (via SVG2Lottie)

  → support a progressive “static-first, then dynamic” training strategy

## multimodal generation (caption)

- BLIP-2 for static graphics captions
- Qwen2.5-VL 32B for temporally aligned animation descriptions after rendering to video.

# Data Curation Pipeline

**rigorous filtering**

1. removing rendering failures and visual anomalies

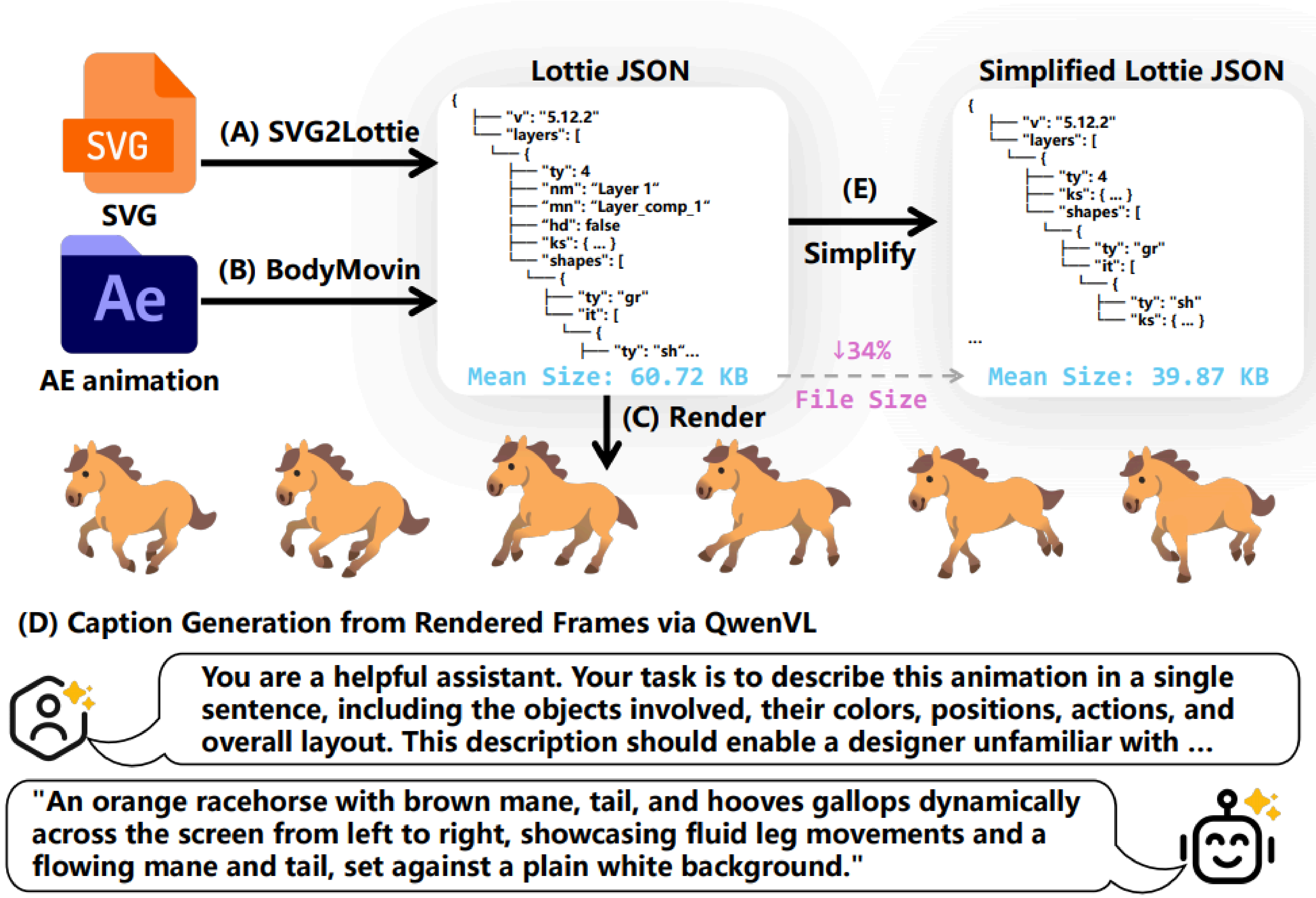
- ex) blank frames, misalignment, flickering

2. simplifying Lottie JSON by removing redundant fields

- ex) comments, unused properties, debug metadata

=> preserving rendering fidelity, reduces sequence length by 34% without affecting quality

## Data Curation Pipeline

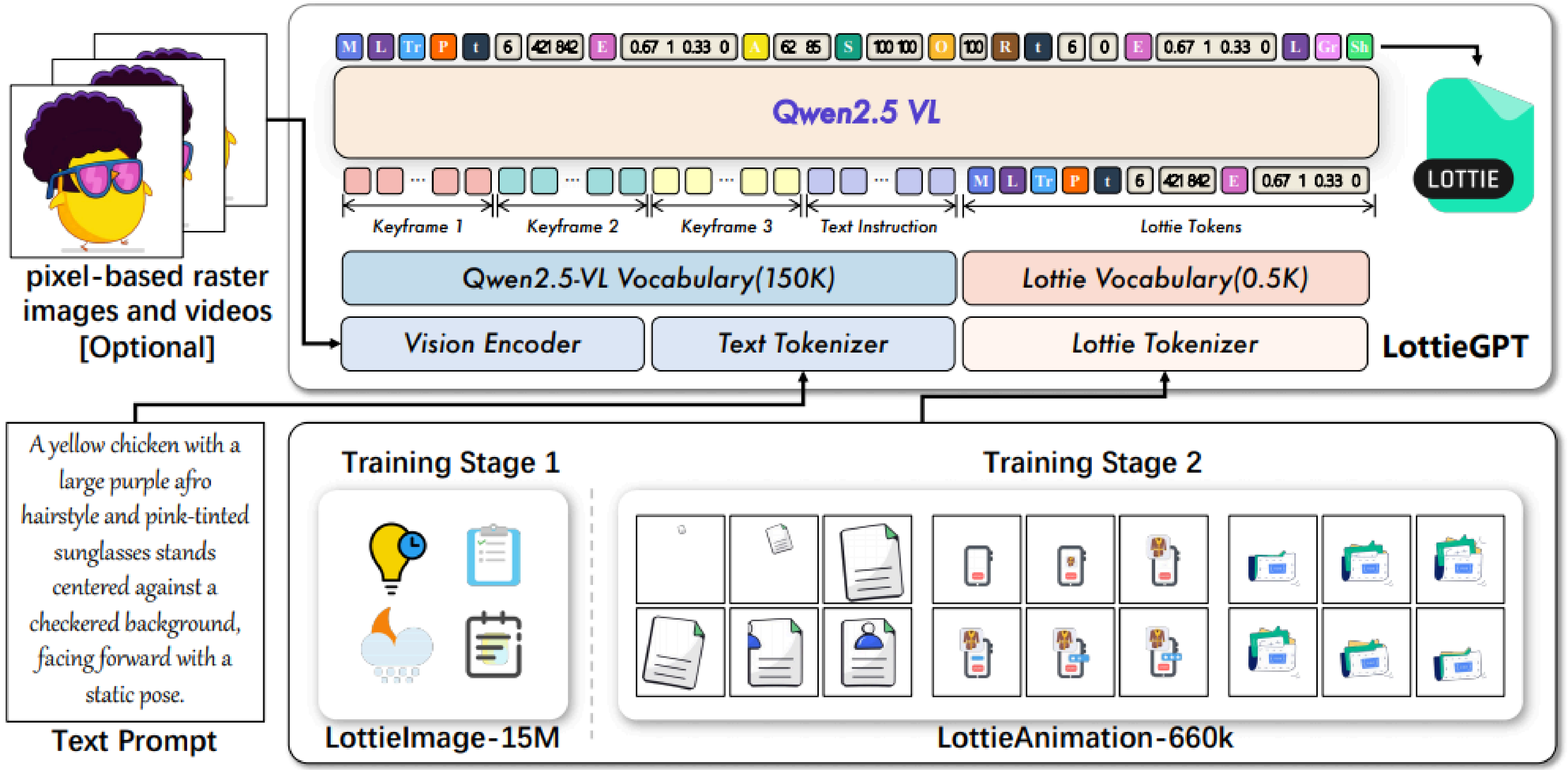



1. collected **10M SVG resources** and **660K After Effects (AE) animation resources** from the internet
2. converted them to **Lottie Json format**
3. **filtered** them using simplification algorithms
4. used QwenVL to generate **text labels** for vector graphics and vector animations

# Methods

## LottieGPT Framework

- **Quen 2.5VL** architecture → native vector animation generation
- extend the model vocabulary with specialized **Lottie tokens**
- design a compact animation tokenization scheme




1) **a Lottie Tokenizer** :
JSON-based animations → discrere token sequences
2) **a vision -language backbone** :
processes multimodal inputs
3) **two-stage training strategy**
static graphics → dynamic animations

=> ‘text descriptions, reference images, or keyframe videos’ → Lottie token sequences

# Methods - Compact Animation Tokenization

**Autoregressive models** : process information as discrete token sequences

→ compact tokenization is crucial!

<prior autoregressive vector graphics generation work **two main issues.**>

1. decomposing SVGs into basic atomic commands → **loses semantic information at the layer group level**
2. cannot represent vector animations, **solely on static vector frame images**

“a specialized **tokenizer** for Lottie animations “

- hierarchically encodes layers, shapes, and temporal keyframes
- exploits the structured nature of Lottie JSON → achieve superior compression

# Methods - Compact Animation Tokenization

## 1. Hierarchical Structure Encoding

Lottie animation  hierarchical structure : Animation Meta → Assets → Layers → Shapes → Properties

1. **Animation Meta Encoding**

   : encode global animation metadata using a compact representation

   <|M|><|v|>"5.9.5"<|fr|>30<|ip|>0<|op|>90<|w|>512<|h|>512<|ddd|>0

   (version, frame rate, inpoint, out-point, dimensions, and 3D flag)

2. **Layer Encoding**

   : Each layer is encoded with its type, transform, and child elements

   <|LAYER|><|ty|>4<|ip|>0<|op|>90<|st|>0<|bm|>0<|LAYER_KS|>...

   * (<|LAYER|>, <|ty|>) :  directly correspond to Lottie schema → learn structural patterns

# Methods - Compact Animation Tokenization

## 1. Hierarchical Structure Encoding

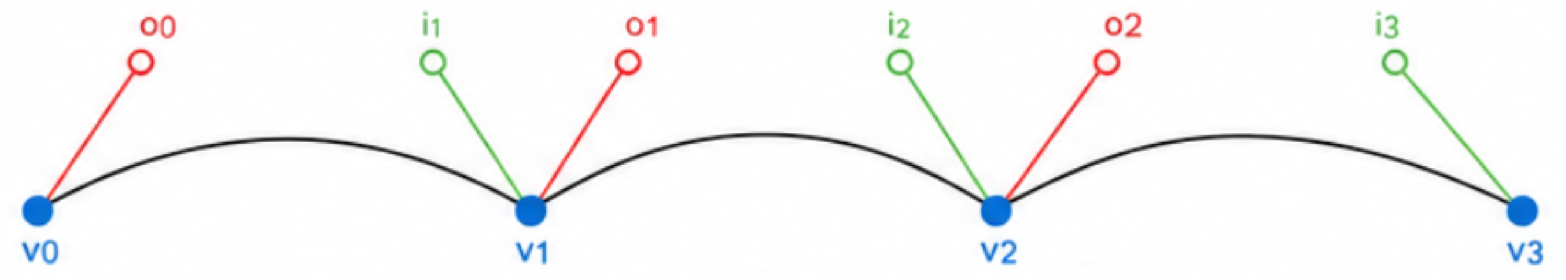
### 3. **Shape Encoding**

: Within each layer, encode geometric shapes using a type-specific format

ex) path shape

<|ITEM_sh|><|KS_STATIC|><|i|>0 0 -10 5<|o|>0 0 10 -5<|v|>100 50 150 75<|c|>

| 요소 | 의미 |
| --- | --- |
| vertices (v) | path control point |
| in-tangent (i) | 들어오는 tangent |
| out-tangent (o) | 나가는 tangent |
| closed flag (c) | path가 닫혔는지 여부 |



# Methods - Compact Animation Tokenization

## 2. Keyframe-Based Motion Compression

→ encode only **keyframes** and **interpolation functions**

- **Unified Property Animation Encoding**

  Each animated property follows the same structural pattern :

  <|PROP_ANIMATED|><|PROP_KF_START|><keyframe_1><keyframe_2>...<keyframe_n><|PROP_KF_END|>

- **Transform Animation**

  five core properties : position (p), anchor point (a), scale (s), rotation (r), opacity (o)

- **Keyframe Structure**

  three essential components:

  1) <|t|>: Time in frames that specifies when this keyframe occurs

  2) Value tokens: Property-specific encoding based on dimensionality

  3) <|ease|>: Cubic Bezier easing function (optional)

# Methods - Compact Animation Tokenization

## 2. Keyframe-Based Motion Compression

**Compression Analysis**

**100-frame** animation → smooth transitions across **6 keyframes**

- easing function <|ease|>

  → critical information for professional animation quality

  → the same keyframes to produce drastically different motion feels (linear, ease-in, bounce, etc.)

**keyframe encoding  Advantages**

1) temporal scalability (compression ratio: 98% for 300 frames/5 keyframes)

2) motion quality preservation (easing curves are first-class primitives)

3) editability (individual keyframes can be modified)

4) resolution independence

5) semantic integrity  (shapes and layers are encoded as complete hierarchical units)

# easing function

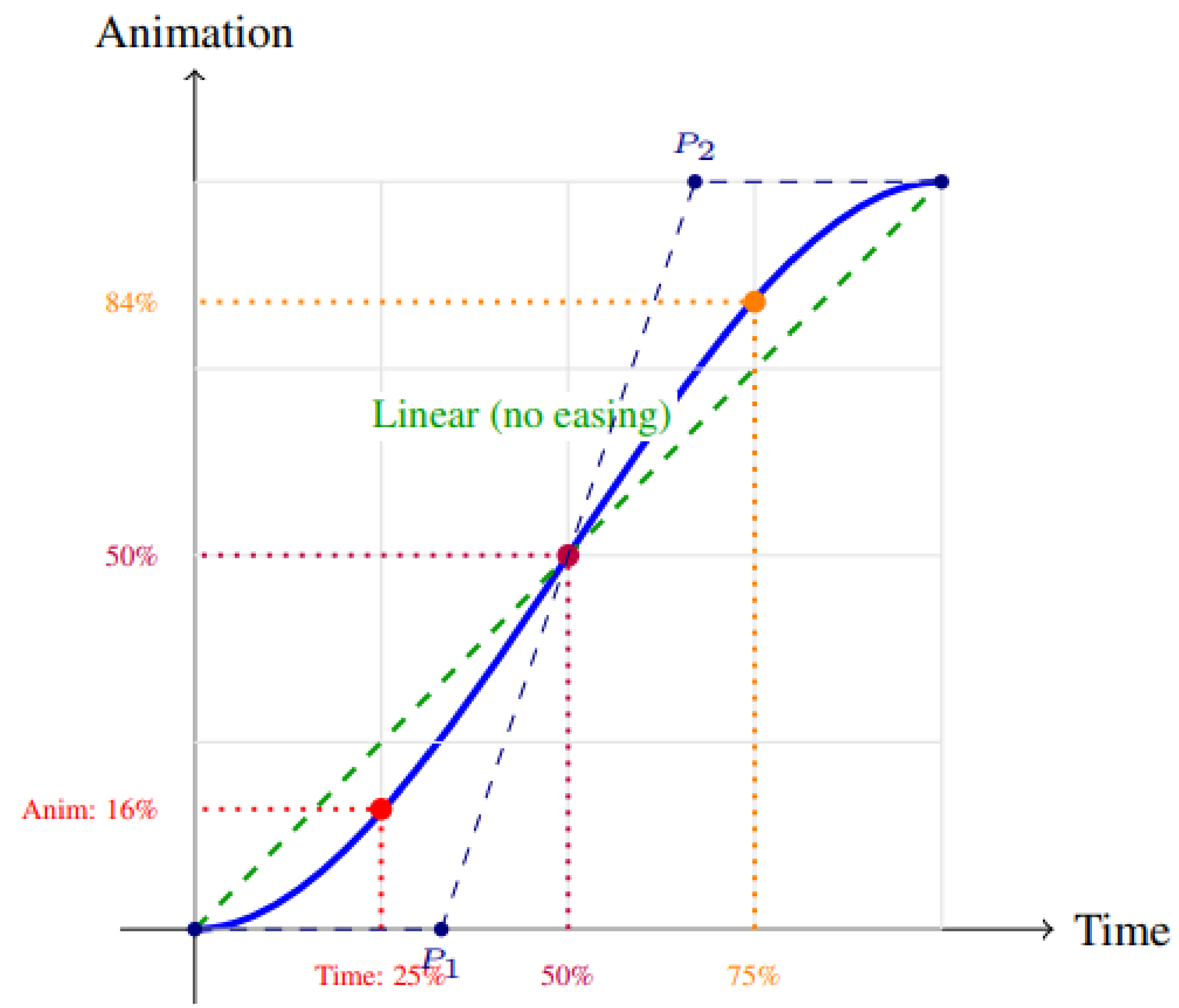



Figure 24. Bézier easing curve transforms time progress (x-axis) into animation progress (y-axis). At 25% time, animation has only progressed 16% (slow start); at 75% time, animation has reached 84% (slow end).

“현재 keyframe 값에서 다음 keyframe 값으로 이동할 때, 시간에 따라 얼마나 이동할지를 결정하는 함수”

→ 시간의 흐름(x축)을 실제 애니메이션 진행률(y축)로 변환

ex)
frame 0 : y = 0
frame 30 : y = 300

<linear>

| time | y |
|---|---|
| 0 | 0 |
| 15 | 150 |
| 30 | 300 |

<easing>

| time | y |
|---|---|
| 0 | 0 |
| 5 | 20 |
| 15 | 150 |
| 25 | 280 |
| 30 | 300 |

# Methods - Compact Animation Tokenization

## 2. Keyframe-Based Motion Compression

**Tokenization**

: metadata, assets, layers, and shapes → **special tokens**

→ traverse the Lottie JSON tree depth-first

- "layers": [...] → <|LAYER|>

- static (<| STATIC|>) or animated (<| ANIMATED|>)

  "a": 0 → static

- property-specific compression

  (colors→hex,motion→Bezier)

**Detokenization**

: token sequence → **JSON**

→ reconstructs the JSON via recursive parsing, achieving **lossless** roundtrip

# Methods - Compact Animation Tokenization

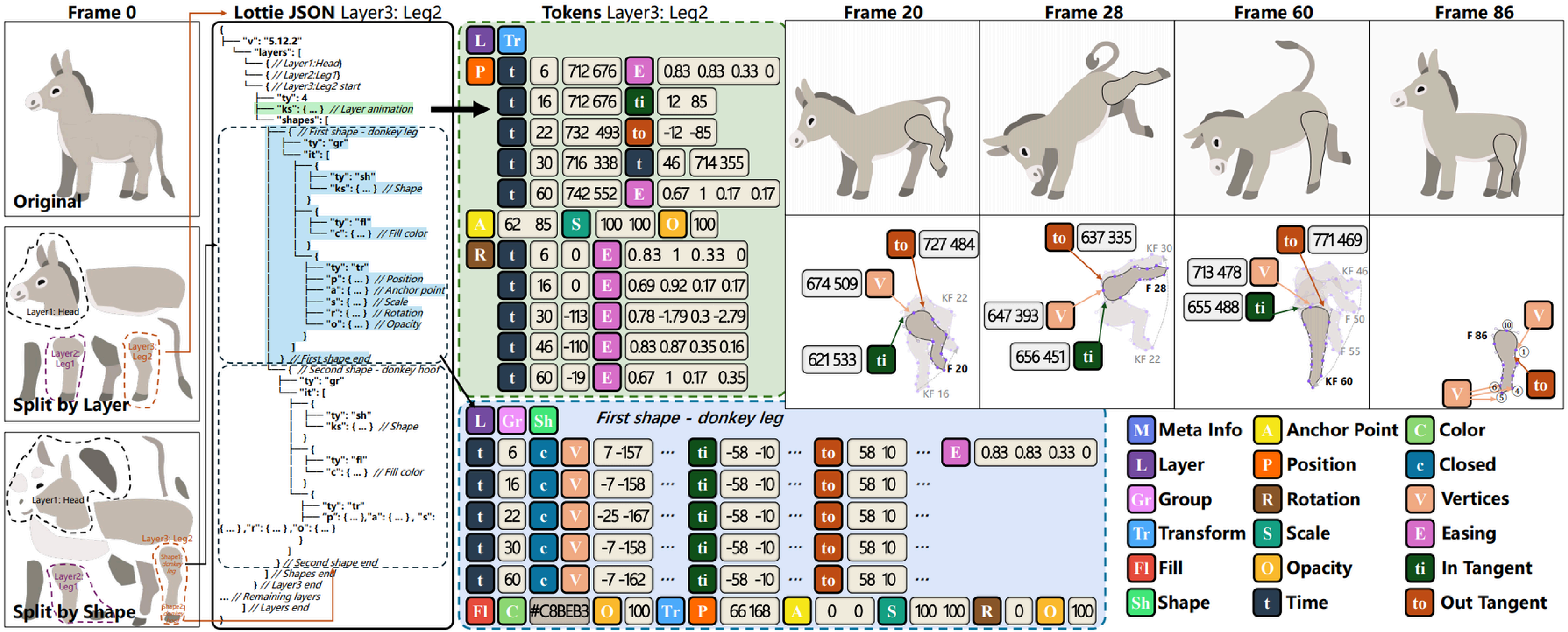



Figure 4. Unlike raster pixel-based videos or frame-by-frame saved SVGs, the Lottie Tokenizer only stores **keyframes** and **interpolation** methods, which significantly reduces the number of tokens required to represent an animation. In the figure, ***KF*** denotes keyframes, while ***F*** represents frames obtained through easing-based animation interpolation.

# Methods - Static-to-Dynamic Training

**two stage training approach**

1. Static Vector Graphics

train on static Lottie images (converted from SVG) (50% text-only data, 50% multimodal data)

→ teaches **fundamental vector composition** (shapes, fills, strokes, transforms, and hierarchical structure)

2. Vector Animation

introduce **temporal dynamics** using Lottie animations

(34% text-only, 33% text + first-frame image, 33% text + video (keyframes))

**Why Two Stages?**

- joint training on mixed static+dynamic data showed **unstable convergence**
- lottie animation contains many tokens

  → difficult for the model to simultaneously learn **spatial composition** and **temporal coordination**

# Methods - Static-to-Dynamic Training

## Training Objective

cross-entropy loss on next-token prediction

$$\mathcal{L} = -\sum_{i=1}^{N} \log P(t_i \mid t_{<i}, \mathbf{c})$$

- ti : i-th token, t<i represents all previous tokens
- c : multimodal conditioning (text + image/video features)

# LottieBench

## Evaluation Data

- For animation generation

  test set : Simple (150 samples), Medium (40 samples), and Complex (40 samples)

  → three difficulty levels by token count

- For static graphics generation

  400 randomly selected unseen samples (typically under 200 tokens)

## Evaluation Metrics

- **Visual-Level Metrics** : assess perceptual quality (LPIPS , SSIM , CLIP, DINO)

- **Structural-Level Metrics** : evaluate JSON structure consistency

1. Flatten JSON hierarchy : depth-first traversal, JSON tree → (path, value), flatten!

$$\Phi(\mathcal{J}) = \{(k, v) \mid k \in \mathcal{K}(\mathcal{J})\}$$

k : a hierarchical path

ex) layers[0].shapes[1].ty : ‘the type of the second shape in the first layer’

# LottieBench

## Evaluation Metrics

- **Structural-Level Metrics** : evaluate JSON structure consistency

2. Compare **key sets.**

→ Partition keys into common (K^c), missing (K^m), and extra (K^e)

$$\mathcal{K}^{c} = \mathcal{K}^{\mathrm{gt}} \cap \mathcal{K}^{\mathrm{pred}}, \quad \mathcal{K}^{m} = \mathcal{K}^{\mathrm{gt}} \setminus \mathcal{K}^{\mathrm{pred}}, \quad \mathcal{K}^{e} = \mathcal{K}^{\mathrm{pred}} \setminus \mathcal{K}^{\mathrm{gt}}$$

→ Compute Key-F1 to measure topology correctness:

$$\text{Key-}F_1 = \frac{2|\mathcal{K}^{c}|}{|\mathcal{K}^{\mathrm{gt}}| + |\mathcal{K}^{\mathrm{pred}}|}$$

3. Measure **value consistency**

$$\delta_k = \mathbb{1}\{v_k^{gt} = v_k^{pred}\}$$

$$\text{ValueMatch} = \frac{1}{|\mathcal{K}^{c}|} \sum_{k \in \mathcal{K}^{c}} \delta_k \qquad (5)$$

$$\text{NumericMAE} = \frac{1}{|\mathcal{N}|} \sum_{k \in \mathcal{N}} |v_k^{\mathrm{gt}} - v_k^{\mathrm{pred}}| \qquad (6)$$

where N ⊆ Kc contains numeric keys.

# LottieBench

## Evaluation Metrics

- **Structural-Level Metrics** : evaluate JSON structure consistency

  Overall score : Combine topology and content with 7:3 weighting

$$\mathrm{JsonStructSim} = 0.7 \cdot \mathrm{Key}\text{-}F_1 + 0.3 \cdot \mathrm{ValueMatch} \qquad (7)$$

- **Semantic-Level Metrics**

  evaluate alignment between **generated content** and **input text prompts** using **CLIP**

  - static image generation tasks : CLIP textimage similarity ( input prompt <-> rendered image)

  - animation generation tasks : average CLIP 6score across all frames

# Results

## Baselines

**vector graphics generation task**

<-> state-of-the-art SVG generation methods OmniSVG and StarVector

**vector animation generation task**

<-> stateof-the-art video generation models including Sora 2 , Kling , and Veo 3.1

<-> state-of-the-art commercial models GPT-5, Claude Sonnet 4.5, Gemini 2.5 Pro, Qwen3-Max, DeepSeek-V3.1

* vector animation generation methods LINR-bridge and AniClipart, are based on LiveSketch

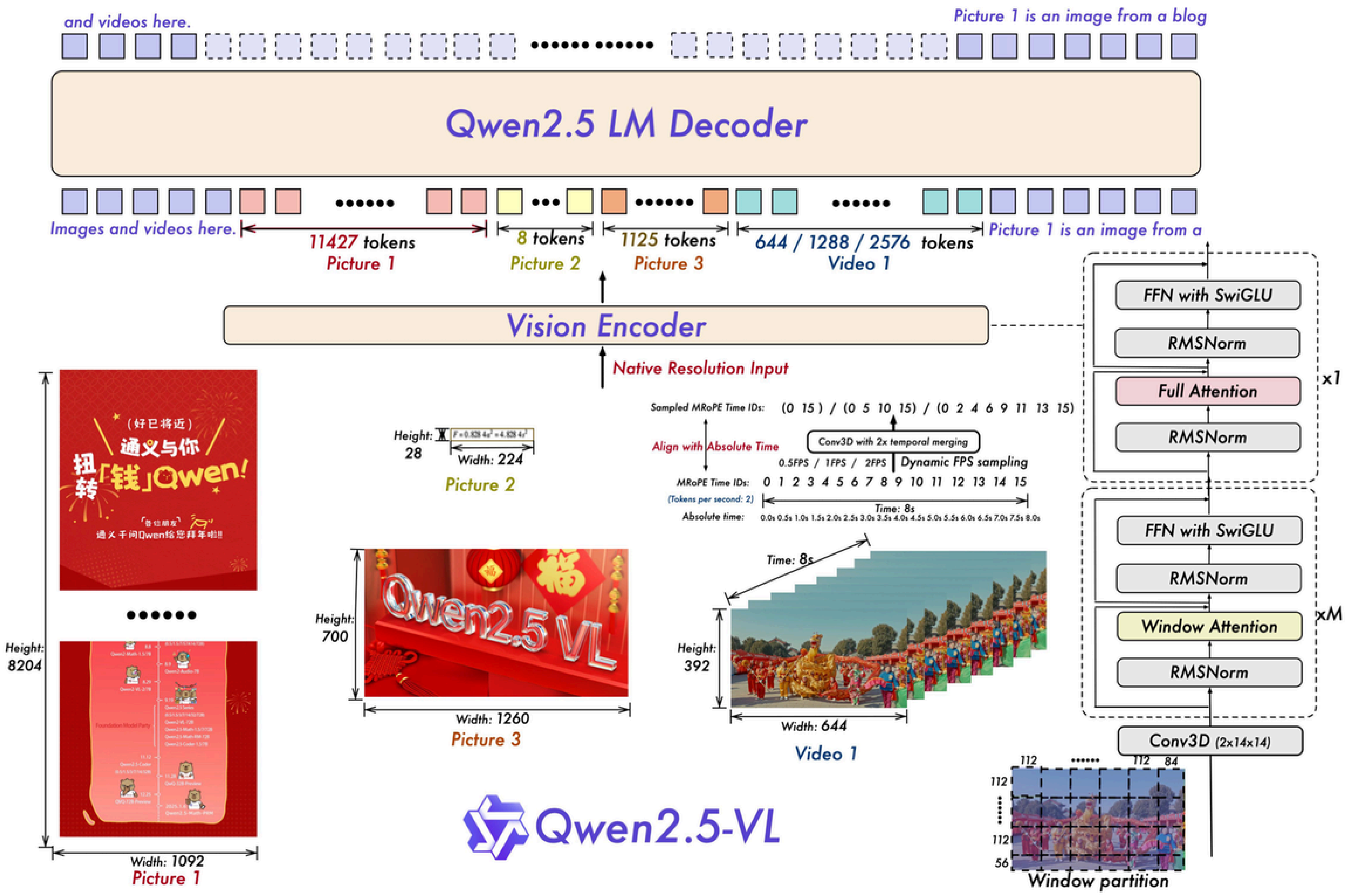
→ cannot handle the complex structure of Lottie animations

# Results

## training details

follow Qwen2.5-VL's hyperparameters

| 항목 | 설정 |
|---|---|
| vision encoder | frozen |
| trainable part | MLP adapter + LLM |
| learning rate | (1 \times 10^{-6}) |
| max sequence length | 40K tokens |
| GPU | 8×H20 140GB |
| precision | bfloat16 |
| optimization | DeepSpeed ZeRO-3 |
| training | stage당 20 epoch |



# Results

## Evaluations and Comparisons

Static Vector Graphics Task

Table 2. Lottie image and animation generation with Text-Only, Text+Image input.

| Method | Model/Approach | Visual Layer | | | | Structural Layer | Semantic Layer | Valid Rate |
|---|---|---|---|---|---|---|---|---|
| | | CLIP↑ | SSIM↑ | LPIPS↓ | DINOv2↑ | JSON↑ | CLIP-T↑ | (%) |
| *Text-Only Input to Static Vector Graphics* | | | | | | | | |
| Text2SVG | OmniSVG-7B | 0.8321 | 0.5627 | 0.5123 | 0.7274 | N/A | 0.2676 | N/A |
| **Ours (Text Only)** | **LottieGPT-7B-Stage1** | **0.9331** | **0.8103** | **0.1755** | **0.8572** | **0.8239** | **0.2893** | **98.25%** |
| *Text+Image Input to Static Vector Graphics* | | | | | | | | |
| Image2SVG | StarVector-8B | 0.7662 | 0.3851 | 0.4647 | 0.5286 | N/A | 0.2454 | N/A |
| | OmniSVG-7B | 0.9002 | 0.7051 | 0.2507 | 0.8481 | N/A | 0.2772 | N/A |
| **Ours (Text+Image)** | **LottieGPT-7B-Stage1** | **0.9201** | **0.8151** | **0.1941** | **0.8514** | **0.8472** | **0.2871** | **97.50%** |
| *Text-Only Input to Vector Animation* | | | | | | | | |
| Few-shot (3-shot) | GPT-5 | 0.7682 | 0.8232 | 0.2528 | 0.5929 | 0.0358 | 0.2640 | 10.87% |
| | Claude Sonnet 4.5 | 0.8485 | 0.8493 | 0.2496 | 0.7278 | 0.0522 | 0.3115 | 45.22% |
| | Gemini 2.5 Pro | 0.7157 | 0.6574 | 0.2113 | 0.6400 | 0.0403 | 0.2565 | 30.00% |
| | DeepSeek-V3.1 | 0.5425 | 0.6390 | 0.1358 | 0.3721 | 0.0399 | 0.1632 | 30.43% |
| | Qwen3-235B | 0.8122 | 0.7973 | 0.3084 | 0.6637 | 0.0151 | 0.3185 | 28.26% |
| Finetuned w. Lottie Json | Qwen2.5-VL 7B | 0.7456 | 0.7156 | 0.1714 | 0.4789 | 0.0089 | 0.2236 | 22.61% |
| **Ours (Text Only)** | **LottieGPT-7B-Stage1** | **0.8156** | **0.8524** | **0.1925** | **0.7715** | **0.2347** | **0.2756** | **78.35%** |
| **Ours (Text Only)** | **LottieGPT-7B-Stage2** | **0.9566** | **0.9776** | **0.0366** | **0.9301** | **0.8062** | **0.2985** | **96.96%** |
| *Text + Image Input to Vector Animation* | | | | | | | | |
| Few-shot (3-shot) | Claude Sonnet 4.5 | 0.8499 | 0.9229 | 0.1273 | 0.7349 | 0.0043 | 0.3416 | 51.74% |
| | Gemini 2.5 Pro | 0.8476 | 0.8311 | 0.2136 | 0.6980 | 0.0037 | 0.2862 | 32.17% |
| | Qwen3-235B | 0.8162 | 0.9251 | 0.1721 | 0.7084 | 0.0028 | 0.2831 | 24.35% |
| Image2Video | Sora2 | 0.8903 | 0.8661 | 0.3004 | 0.8182 | N/A | N/A | N/A |
| | Kling | 0.8661 | 0.8542 | 0.2783 | 0.6261 | N/A | N/A | N/A |
| | Veo3.1 | 0.8487 | 0.5558 | 0.4417 | 0.6658 | N/A | N/A | N/A |
| **Ours (Text+Image)** | **LottieGPT-7B-Stage2** | **0.9743** | **0.9886** | **0.0278** | **0.9472** | **0.9195** | **0.3029** | **97.83%** |

# Results

## Evaluations and Comparisons

Vector Animation Task

Table 2. Lottie image and animation generation with Text-Only, Text+Image input.

| Method | Model/Approach | Visual Layer | | | | Structural Layer | Semantic Layer | Valid Rate |
|---|---|---|---|---|---|---|---|---|
| | | CLIP↑ | SSIM↑ | LPIPS↓ | DINOv2↑ | JSON↑ | CLIP-T↑ | (%) |
| *Text-Only Input to Static Vector Graphics* | | | | | | | | |
| Text2SVG | OmniSVG-7B | 0.8321 | 0.5627 | 0.5123 | 0.7274 | N/A | 0.2676 | N/A |
| **Ours (Text Only)** | **LottieGPT-7B-Stage1** | **0.9331** | **0.8103** | **0.1755** | **0.8572** | **0.8239** | **0.2893** | **98.25%** |
| *Text+Image Input to Static Vector Graphics* | | | | | | | | |
| Image2SVG | StarVector-8B | 0.7662 | 0.3851 | 0.4647 | 0.5286 | N/A | 0.2454 | N/A |
| | OmniSVG-7B | 0.9002 | 0.7051 | 0.2507 | 0.8481 | N/A | 0.2772 | N/A |
| **Ours (Text+Image)** | **LottieGPT-7B-Stage1** | **0.9201** | **0.8151** | **0.1941** | **0.8514** | **0.8472** | **0.2871** | **97.50%** |
| *Text-Only Input to Vector Animation* | | | | | | | | |
| Few-shot (3-shot) | GPT-5 | 0.7682 | 0.8232 | 0.2528 | 0.5929 | 0.0358 | 0.2640 | 10.87% |
| | Claude Sonnet 4.5 | 0.8485 | 0.8493 | 0.2496 | 0.7278 | 0.0522 | 0.3115 | 45.22% |
| | Gemini 2.5 Pro | 0.7157 | 0.6574 | 0.2113 | 0.6400 | 0.0403 | 0.2565 | 30.00% |
| | DeepSeek-V3.1 | 0.5425 | 0.6390 | 0.1358 | 0.3721 | 0.0399 | 0.1632 | 30.43% |
| | Qwen3-235B | 0.8122 | 0.7973 | 0.3084 | 0.6637 | 0.0151 | 0.3185 | 28.26% |
| Finetuned w. Lottie Json | Qwen2.5-VL 7B | 0.7456 | 0.7156 | 0.1714 | 0.4789 | 0.0089 | 0.2236 | 22.61% |
| **Ours (Text Only)** | **LottieGPT-7B-Stage1** | **0.8156** | **0.8524** | **0.1925** | **0.7715** | **0.2347** | **0.2756** | **78.35%** |
| **Ours (Text Only)** | **LottieGPT-7B-Stage2** | **0.9566** | **0.9776** | **0.0366** | **0.9301** | **0.8062** | **0.2985** | **96.96%** |
| *Text + Image Input to Vector Animation* | | | | | | | | |
| Few-shot (3-shot) | Claude Sonnet 4.5 | 0.8499 | 0.9229 | 0.1273 | 0.7349 | 0.0043 | 0.3416 | 51.74% |
| | Gemini 2.5 Pro | 0.8476 | 0.8311 | 0.2136 | 0.6980 | 0.0037 | 0.2862 | 32.17% |
| | Qwen3-235B | 0.8162 | 0.9251 | 0.1721 | 0.7084 | 0.0028 | 0.2831 | 24.35% |
| Image2Video | Sora2 | 0.8903 | 0.8661 | 0.3004 | 0.8182 | N/A | N/A | N/A |
| | Kling | 0.8661 | 0.8542 | 0.2783 | 0.6261 | N/A | N/A | N/A |
| | Veo3.1 | 0.8487 | 0.5558 | 0.4417 | 0.6658 | N/A | N/A | N/A |
| **Ours (Text+Image)** | **LottieGPT-7B-Stage2** | **0.9743** | **0.9886** | **0.0278** | **0.9472** | **0.9195** | **0.3029** | **97.83%** |

# Results

## Evaluations and Comparisons

Vector Animation Task

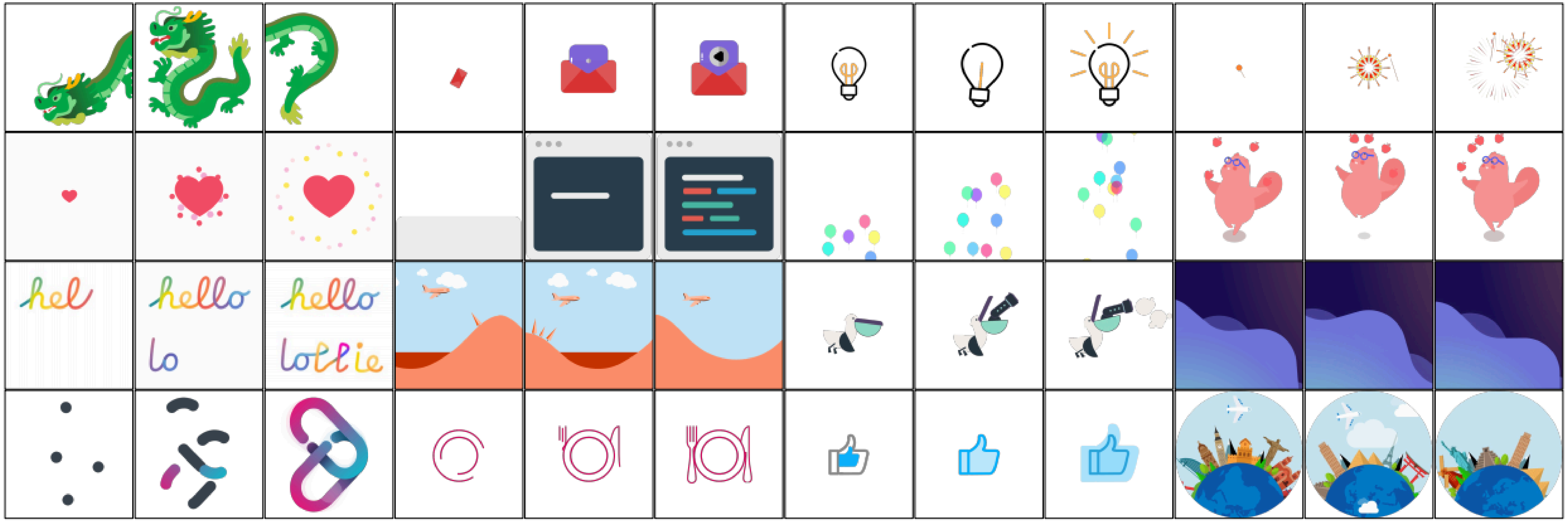



Figure 5. Lottie animations generated by LottieGPT.

# Results

## Evaluations and Comparisons

Ablation Study

Table 2. Lottie image and animation generation with Text-Only, Text+Image input.

| Method | Model/Approach | Visual Layer | | | | Structural Layer | Semantic Layer | Valid Rate |
|---|---|---|---|---|---|---|---|---|
| | | CLIP↑ | SSIM↑ | LPIPS↓ | DINOv2↑ | JSON↑ | CLIP-T↑ | (%) |
| *Text-Only Input to Static Vector Graphics* | | | | | | | | |
| Text2SVG | OmniSVG-7B | 0.8321 | 0.5627 | 0.5123 | 0.7274 | N/A | 0.2676 | N/A |
| **Ours (Text Only)** | **LottieGPT-7B-Stage1** | **0.9331** | **0.8103** | **0.1755** | **0.8572** | **0.8239** | **0.2893** | **98.25%** |
| *Text+Image Input to Static Vector Graphics* | | | | | | | | |
| Image2SVG | StarVector-8B | 0.7662 | 0.3851 | 0.4647 | 0.5286 | N/A | 0.2454 | N/A |
| | OmniSVG-7B | 0.9002 | 0.7051 | 0.2507 | 0.8481 | N/A | 0.2772 | N/A |
| **Ours (Text+Image)** | **LottieGPT-7B-Stage1** | **0.9201** | **0.8151** | **0.1941** | **0.8514** | **0.8472** | **0.2871** | **97.50%** |
| *Text-Only Input to Vector Animation* | | | | | | | | |
| Few-shot (3-shot) | GPT-5 | 0.7682 | 0.8232 | 0.2528 | 0.5929 | 0.0358 | 0.2640 | 10.87% |
| | Claude Sonnet 4.5 | 0.8485 | 0.8493 | 0.2496 | 0.7278 | 0.0522 | 0.3115 | 45.22% |
| | Gemini 2.5 Pro | 0.7157 | 0.6574 | 0.2113 | 0.6400 | 0.0403 | 0.2565 | 30.00% |
| | DeepSeek-V3.1 | 0.5425 | 0.6390 | 0.1358 | 0.3721 | 0.0399 | 0.1632 | 30.43% |
| | Qwen3-235B | 0.8122 | 0.7973 | 0.3084 | 0.6637 | 0.0151 | 0.3185 | 28.26% |
| Finetuned w. Lottie Json | Qwen2.5-VL 7B | 0.7456 | 0.7156 | 0.1714 | 0.4789 | 0.0089 | 0.2236 | 22.61% |
| **Ours (Text Only)** | **LottieGPT-7B-Stage1** | **0.8156** | **0.8524** | **0.1925** | **0.7715** | **0.2347** | **0.2756** | **78.35%** |
| **Ours (Text Only)** | **LottieGPT-7B-Stage2** | **0.9566** | **0.9776** | **0.0366** | **0.9301** | **0.8062** | **0.2985** | **96.96%** |
| *Text + Image Input to Vector Animation* | | | | | | | | |
| Few-shot (3-shot) | Claude Sonnet 4.5 | 0.8499 | 0.9229 | 0.1273 | 0.7349 | 0.0043 | 0.3416 | 51.74% |
| | Gemini 2.5 Pro | 0.8476 | 0.8311 | 0.2136 | 0.6980 | 0.0037 | 0.2862 | 32.17% |
| | Qwen3-235B | 0.8162 | 0.9251 | 0.1721 | 0.7084 | 0.0028 | 0.2831 | 24.35% |
| Image2Video | Sora2 | 0.8903 | 0.8661 | 0.3004 | 0.8182 | N/A | N/A | N/A |
| | Kling | 0.8661 | 0.8542 | 0.2783 | 0.6261 | N/A | N/A | N/A |
| | Veo3.1 | 0.8487 | 0.5558 | 0.4417 | 0.6658 | N/A | N/A | N/A |
| **Ours (Text+Image)** | **LottieGPT-7B-Stage2** | **0.9743** | **0.9886** | **0.0278** | **0.9472** | **0.9195** | **0.3029** | **97.83%** |

# Conclusion

- LottieGPT : the first framework for autoregressively generating editable vector animations from text, images, or keyframes.
- Lottie Tokenizer for hierarchical primitives and keyframes
- constructing a 660K animation dataset
- fine-tuning Qwen2.5-VL via ”static-to-dynamic” training

=> resolution-independent, compact, and editable outputs

=> editable outputs, achieving state-of-the-art performance in both vector animation and SVG generation

# Thank You For Listening

STUDENT
Won Jiwoo

AUTHOR
Junhao Chen